

DATA-260 Homework 2

Pragnaya Priyadarshini Repository: <https://github.com/pragnayap/data260-3170>

Tag for this submission: [7ce961d73d28169da11a24dfe760562da7ee5c50](#)

Section 0 - Personal configuration

Value	Definition	My Configuration
SID4	Last four digits of SJSU Student ID	3170
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8470
PREFIX	"s" + SID4	s3170
SEED	SID4	3170
VERIFY_SEED	$260000 + \text{SID4}$	263170
DOMAIN_ID	$\text{SID4} \bmod 8$	2 - Municipal Transit Incidents

Hardware: MacBook Pro, Intel Core i9-9880H @ 2.30 GHz, 16 cores, 16 GB RAM, macOS 15.7.9 (x86_64).

Local model: qwen3:8b via Ollama, temperature 0.0, reasoning disabled, num_ctx 2048.

All model calls route through the `src/model_client.py` adapter carried over from HW1.

Tagged commit hash: hw2

On hardware: this is an Intel Mac with no GPU acceleration for inference, so qwen3:8b runs on CPU at roughly 2 tokens/second. A warm model call costs ~9 seconds and a cold one ~26 seconds, because Ollama unloads the model between processes. Every latency figure in this report should be read against that baseline, and it is why all 75 experiment runs were executed inside a single process.

Part 1 - HTML & CSS

Requirement: style the HW1 entity list/form so it remains usable at 375px width, and provide visible loading, empty, and error states.

- Styles live in `styles.css`

Responsive layout

The breakpoint is 480px, which covers the required 375px. Below it the page drops to a single column, the search row stacks, every button goes full width, and the incident header wraps.

```

/* --- HW2: usable down to 375px --- */
@media (max-width: 480px) {
  body { margin: 0; padding: 12px; }
  h1 { font-size: 24px; margin-bottom: 20px; }
  form, .panel { padding: 18px 15px; }
  .search-row { flex-direction: column; }
  .search-row button,
  form > button[type="submit"],
  .actions button,
  .state-error button {
    width: 100%;
    margin: 0;
  }
  .state-error button { margin-top: 12px; }
  .actions button { padding: 13px 16px; }
  .actions { flex-direction: column; }
  .incident-head { flex-direction: column; gap: 2px; }
  #routeId::placeholder { font-size: 15px; }
}

```

One defect worth calling out, because it is the failure mode a 375px layout must avoid. Incident text is user-submitted, so a hyphenated route id or a long reference number has no break opportunity and pushes the page into horizontal scroll. There were no wrapping rules in the original stylesheet at all. Fixed with:

```

/* --- HW2: Incident List --- */
ul#incidentList { list-style: none; margin: 15px 0 0; padding: 0; }
ul#incidentList li {
  border: 1px solid var(--line);
  border-left: 4px solid var(--brand);
  border-radius: 6px;
  padding: 12px 15px;
  margin-bottom: 10px;
  overflow-wrap: anywhere;
}

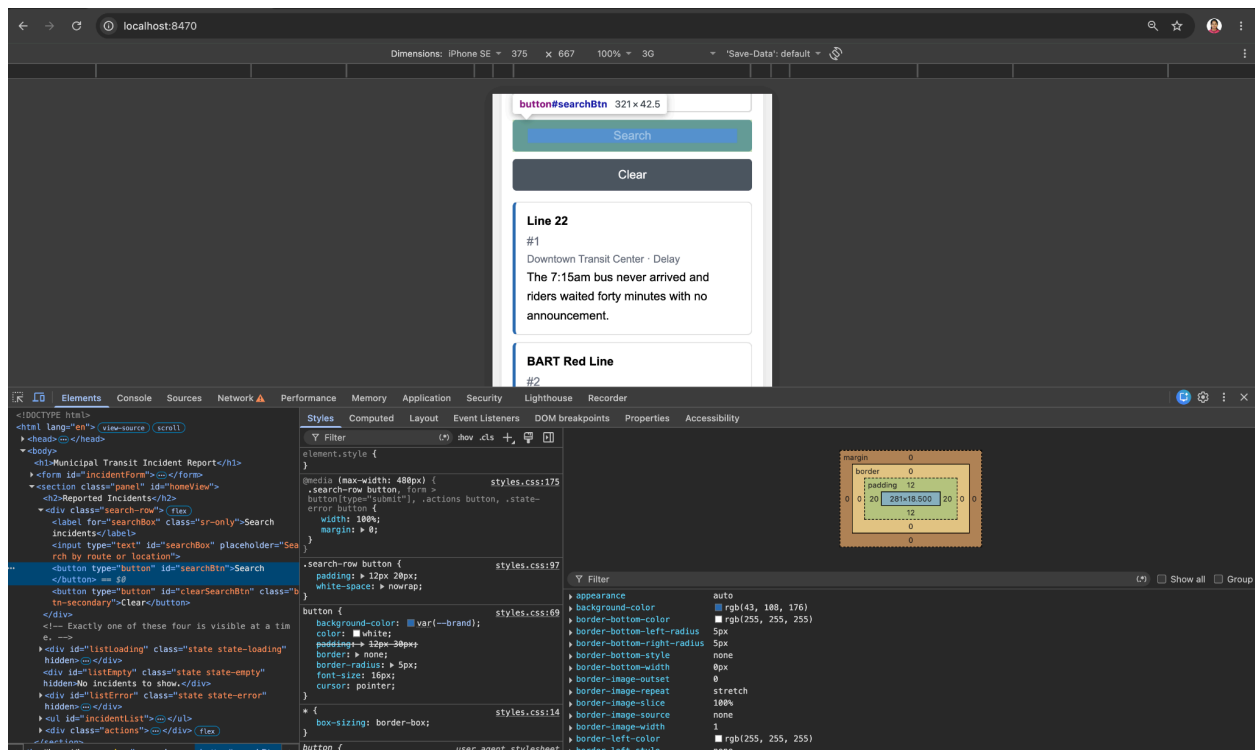
```

SCREENSHOT 1.1- the page at 375px width , showing the stacked layout and a long route id wrapping inside its card rather than overflowing.

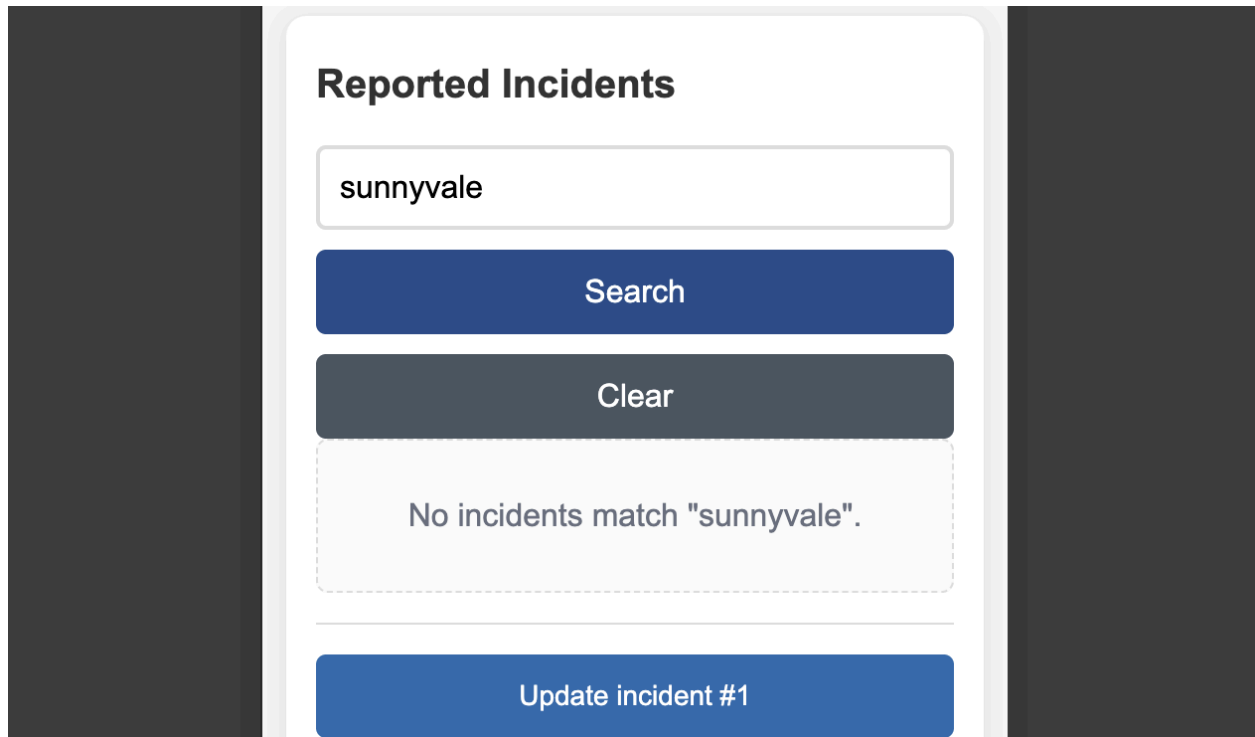

```
const showState = (which, detail = "") => {
  const panes = {
    loading: el("listLoading"),
    empty: el("listEmpty"),
    error: el("listError"),
    list: el("incidentList"),
  };
  Object.entries(panes).forEach(([name, node]) => {
    node.hidden = name !== which;
  });
  if (which === "error") {
    el("listErrorDetail").textContent = detail;
  }
};
```

The empty state distinguishes "nothing exists yet" from "nothing matched your search", and the error state carries the server's message plus a retry button.

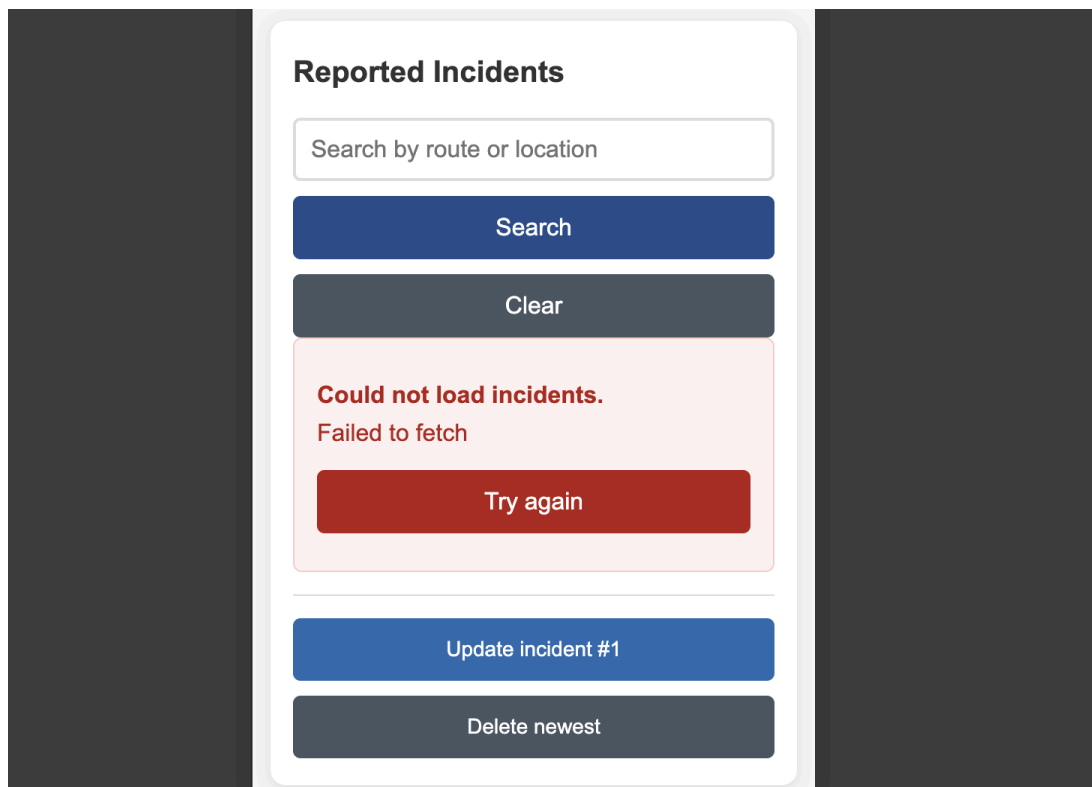
SCREENSHOT 1.2 - the loading state



SCREENSHOT 1.3 - the empty state, produced by searching for a term that matches nothing.



SCREENSHOT 1.4 - the error state, produced by stopping the server and clicking Search.



Part 2 - FastAPI

Implemented in code/web_application/app.py, served on **PORT_BASE 8470**.

2.1 Add a record, then redirect to the home view

```
@app.post("/api/incidents", response_model=TransitIncident, status_code=201)
async def create_incident(incident_data: TransitIncidentCreate, response: Response):
    """Add a record. The client redirects to the list view once this returns."""
    no_store(response)

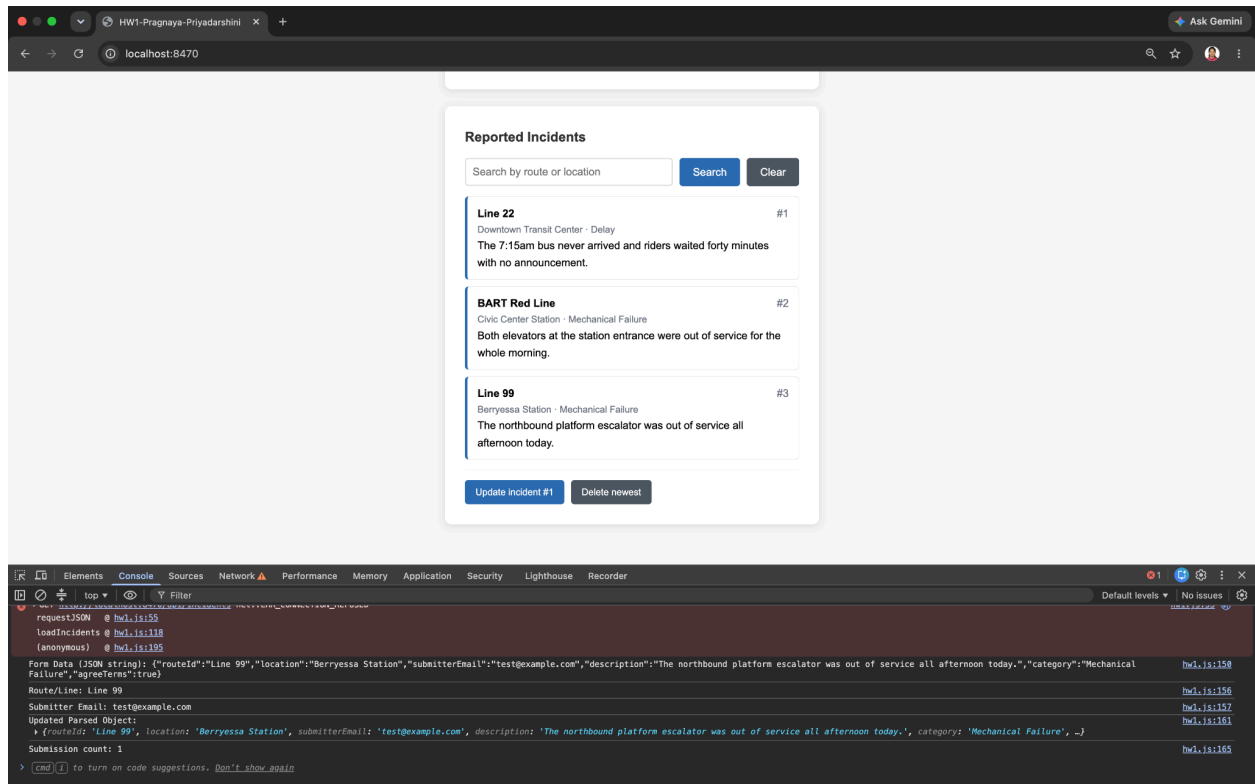
    # Max+1 rather than len+1: after a delete, len+1 can collide with a live id.
    new_id = max((i.id for i in incidents), default=0) + 1

    incident = TransitIncident(id=new_id, **incident_data.model_dump())
    incidents.append(incident)
    return incident
```

The client submits, then returns to the list view with the new record present:

```
const { agreeTerms, ...incidentPayload } = parsed;
await requestJSON(API, { method: "POST", body: JSON.stringify(incidentPayload) });
document.getElementById("incidentForm").reset();
el("searchBox").value = "";
await loadIncidents(); // redirect to home view
el("homeView").scrollIntoView({ behavior: "smooth" });
...
```

SCREENSHOT 2.1 - the filled form, and below it the list view showing the new record.



2.2 Update the record with ID 1

```
@app.put("/api/incidents/{incident_id}", response_model=TransitIncident)
async def update_incident(
    incident_id: int, incident_data: TransitIncidentUpdate, response: Response = None
):
    """Replace the mutable fields of one record, keeping its id."""
    no_store(response)

    existing = find_incident(incident_id)
    updated = TransitIncident(id=existing.id, **incident_data.model_dump())
    incidents[incidents.index(existing)] = updated
    return updated
```

The "Update incident #1" button sends domain-appropriate values for the primary field (routeId) and secondary field (location), then reloads the list.

SCREENSHOT 2.2 - the list before and after, showing record #1 changed to "Line 22 (Rerouted)" at "Downtown Transit Center - Bay 4".

2.3 Delete the record with the highest ID

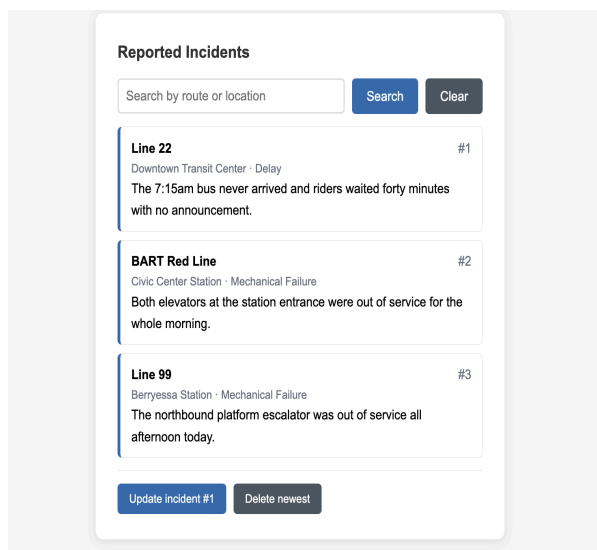
```
@app.delete("/api/incidents/{incident_id}", response_model=TransitIncident)
async def delete_incident(incident_id: int, response: Response = None):
    """Delete one record and return it, so the client can confirm what went."""
    no_store(response)

    incident = find_incident(incident_id)
    incidents.remove(incident)
    return incident
```

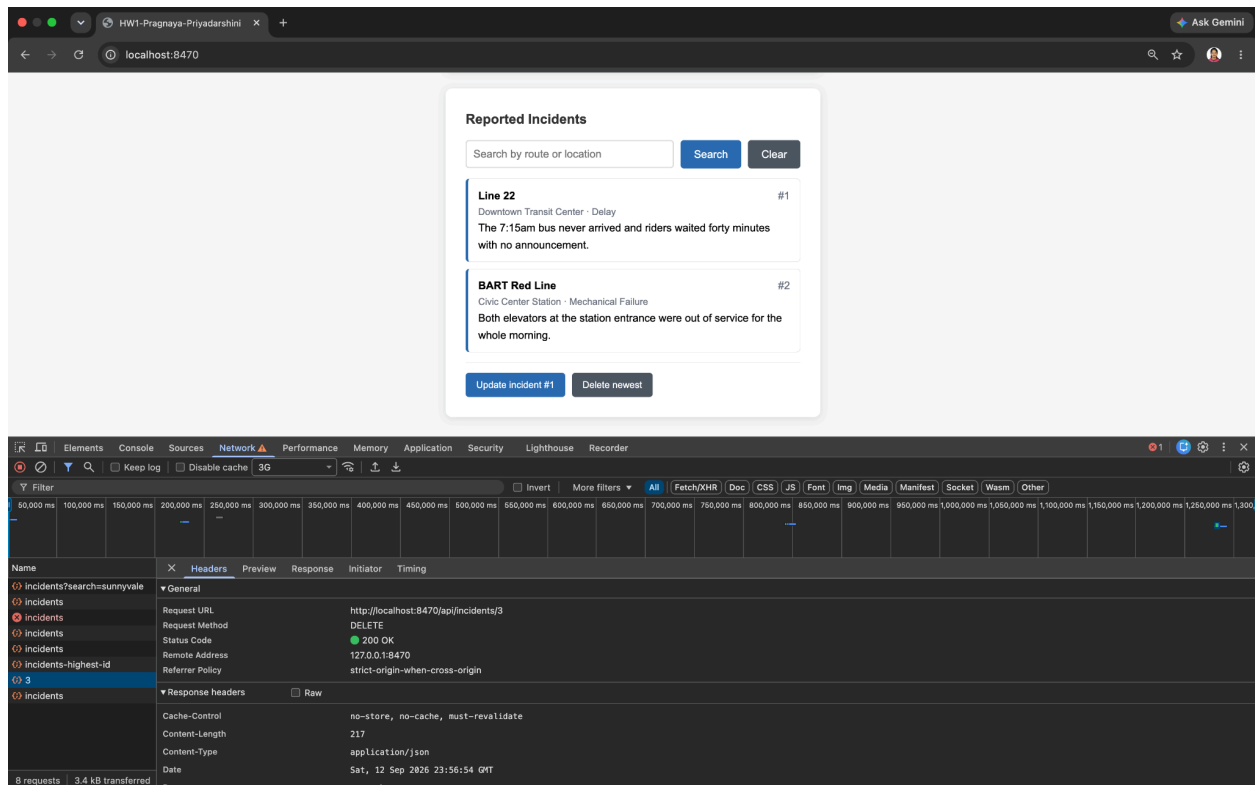
```
// Delete the record with the highest ID, then show the list.
el("deleteHighestBtn").addEventListener("click", async () => {
    showState("loading");
    try {
        const highest = await requestJSON("/api/incidents-highest-id");
        if (highest === null) {
            await loadIncidents();
            return;
        }
        await requestJSON(`${API}/${highest}`, { method: "DELETE" });
        el("searchBox").value = "";
        await loadIncidents();
    } catch (error) {
        showState("error", `Could not delete the newest incident: ${error.message}`);
    }
});
```

SCREENSHOT 2.3 - the list before and after clicking "Delete newest".

The list before:



The list after deleting:



2.4 Search by primary or secondary field

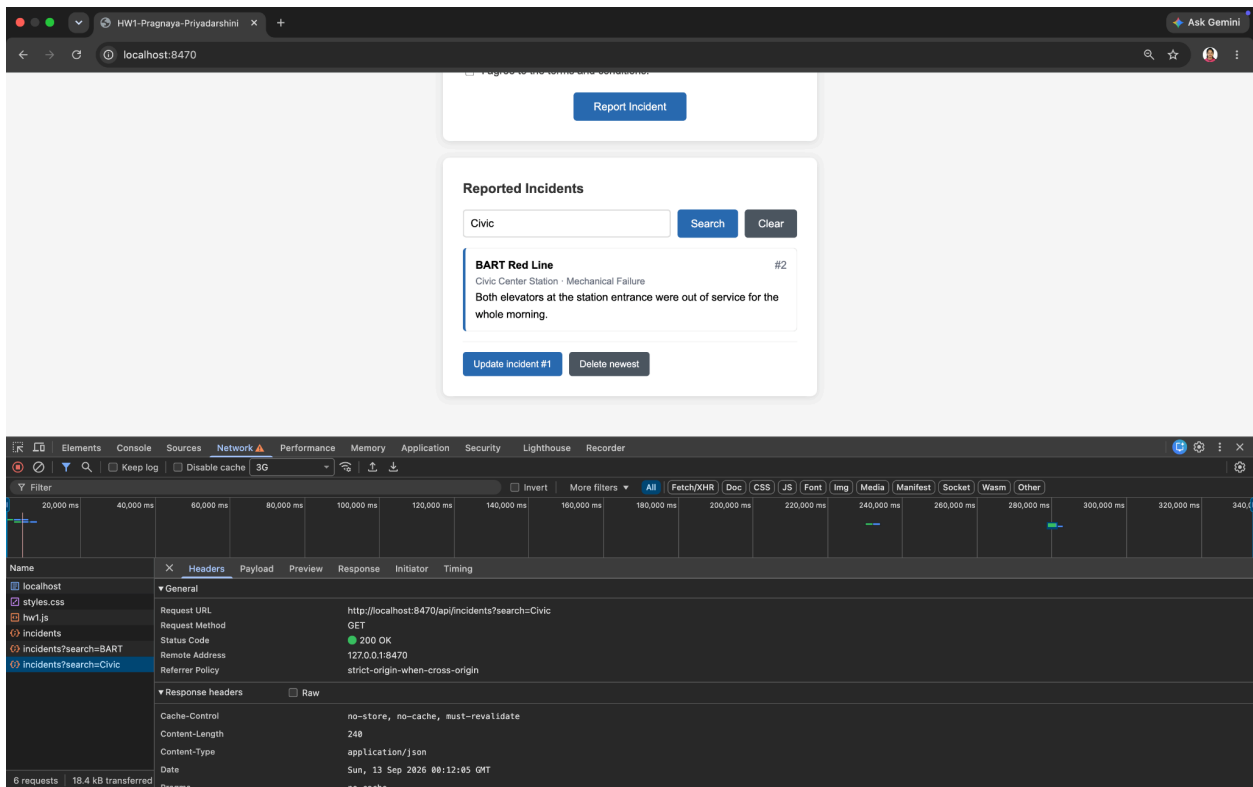
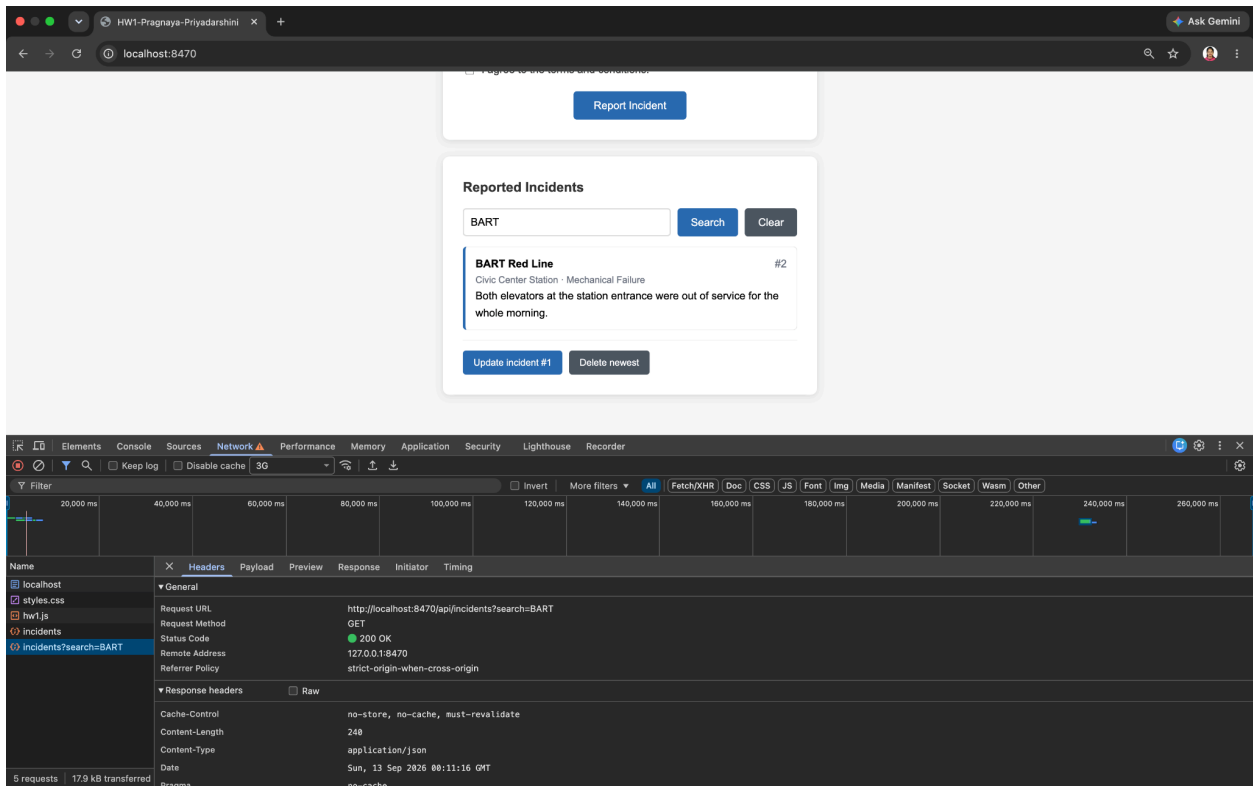
```
@app.get("/api/incidents", response_model=List[TransitIncident])
async def get_incidents(search: Optional[str] = None, response: Response = None):
    no_store(response)

    if not search or not search.strip():
        return incidents

    term = search.strip().lower()
    return [
        incident
        for incident in incidents
        if term in incident.routeId.lower() or term in incident.location.lower()
    ]
```

Case-insensitive substring matches against routeId (primary) and location (secondary).

SCREENSHOT 2.4 - a search term such as “BART” and “civic” entered and the list filtered to matching records only.



Part 3 - Stateful agent graph

Implemented in code/agents_graph.py. HW1's Planner → Reviewer → Finalizer waterfall is replaced by a graph that can loop back for self-correction and stop at a turn ceiling.

```
START -> supervisor -> router_logic --> planner -> supervisor
                                     --> reviewer -> supervisor
                                     --> END
```

Shared state

```
class AgentState(TypedDict):
    """Memory shared by every node. Nodes return only the keys they change."""

    # Inputs (set once, before the graph runs)
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any # ModelClient from src/model_client.py

    # Agent outputs
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]

    # Loop control
    turn_count: int
    turn_ceiling: int

    # Part 4 - schema validation and retry bookkeeping
    valid: bool
    validation_error: str # feedback for the next Planner attempt (" if clean)
    validation_history: List[str]
    planner_attempts: int # every Planner call, whatever sent it back
    schema_retries: int # only the calls that FAILED Pydantic validation
    issue_history: List[str] # every actionable Reviewer objection, in order
```

The two counters are deliberately separate. The Reviewer can send a proposal back for content reasons, which is not a schema failure; conflating them would make the Part 4.3 table measure how picky the Reviewer is rather than how often the schema is satisfied.

The supervisor and the router

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """Update state only -- increment the turn counter. No LLM call here."""
    turn = state["turn_count"] + 1
    print(f"--- NODE: Supervisor (turn {turn}/{state['turn_ceiling']}) ---")
    return {"turn_count": turn}

def router_logic(state: AgentState) -> str:
    if state["turn_count"] >= state["turn_ceiling"]:
        print(f"    router -> END (hit turn ceiling {state['turn_ceiling']})")
        return END

    if not state["valid"]:
        reason = "no proposal yet" if state["planner_attempts"] == 0 else "schema invalid"
        print(f"    router -> planner ({reason})")
        return "planner"

    feedback = state.get("reviewer_feedback") or {}
    if not feedback:
        print("    router -> reviewer (proposal not yet reviewed)")
        return "reviewer"

    if feedback.get("issues"):
        print(f"    router -> planner ({len(feedback['issues'])} reviewer issues)")
        return "planner"

    print("    router -> END (valid and reviewed clean)")
    return END
```

The ceiling check comes first and unconditionally. In any other order, a proposal that keeps failing validation loops forever, because the retry branch fires before the stop condition is ever consulted.

Model-client routing

Both nodes call the adapter, never Ollama or LangChain directly:

```
def check_model_client_routing():
    graph_path = CODE_DIR / "agents_graph.py"
    if not graph_path.exists():
        check("routing:agents_graph_present", False, "file missing")
        return
    source = graph_path.read_text()

    banned = ["ChatOllama(", "make_llm(", "ollama.Client", "langchain_ollama"]
    found = [token for token in banned if token in source]
    check("routing:no_direct_model_calls", not found, f"found {found}" if found else "")

    calls = [ln.strip() for ln in source.splitlines() if ".complete(" in ln]
    routed = [c for c in calls if 'state["llm"].complete(' in c]
    check("routing:all_llm_calls_via_model_client",
          bool(calls) and len(routed) == len(calls),
          f"{len(routed)}/{len(calls)} call sites use state['llm'].complete()")
```

verify_hw02.py checks this statically, since a direct ChatOllama call would still produce correct-looking output:

SCREENSHOT 3.1 - python code/agents_graph.py --stream, showing the node-by-node path: Supervisor → Planner → Supervisor → Reviewer → Supervisor → END.

```
{
  "Service Issue": {
    "summary": "A Line 22 bus was delayed by 40 minutes at Downtown Transit Center with no prior announcement.",
    "validation_history": [],
    "issue_history": [],
    "reviewer_issues": [],
    "latency_ms": 47839
  }
}

[exit] turns=2 cumulative_input_tokens=503 cumulative_output_tokens=55 cumulative_total_tokens=558
(base)
data260-3170 on 7 main [17] via 3 v3.12.13 (.venv) on 1 (us-east-1) took 49s
> python code/agents_graph.py --stream

--- NODE: Supervisor (turn 1/4) ---
router -> planner (no proposal yet)

{supervisor: {
  "turn_count": 1
}}
--- NODE: Planner ---
[turn 1] input_tokens=153 output_tokens=43 total_tokens=196
valid=True

{planner: {
  "planner_proposal": {
    "tags": [
      "Bus Delay",
      "Transit Center",
      "Service Issue"
    ],
    "summary": "A Line 22 bus was delayed 40 minutes at Downtown Transit Center with no prior announcement."
  },
  "valid": true,
  "validation_error": "",
  "validation_history": [],
  "planner_attempts": 1,
  "schema_retries": 0,
  "reviewer_feedback": {}
}}
--- NODE: Supervisor (turn 2/4) ---
router -> reviewer (proposal not yet reviewed)

{supervisor: {
  "turn_count": 2
}}
--- NODE: Reviewer ---
[turn 2] input_tokens=349 output_tokens=8 total_tokens=357
issues=0

{reviewer: {
  "reviewer_feedback": {
    "issues": [],
    "reviewed": true,
    "dropped": 0
  },
  "issue_history": []
}}
--- NODE: Supervisor (turn 3/4) ---
router -> END (valid and reviewed clean)

{supervisor: {
  "turn_count": 3
}}

[exit] turns=2 cumulative_input_tokens=502 cumulative_output_tokens=51 cumulative_total_tokens=553
(base)
data260-3170 on 7 main [17] via 3 v3.12.13 (.venv) on 1 (us-east-1) took 12s
> |
```

Step 6 - testing the correction loop

The spec asks for the Reviewer to be made to always object, so the graph can be watched routing back to the Planner. This is exposed as a flag rather than a temporary source edit, so the demonstration is reproducible and there is nothing to remember to revert:

```
python code/agents_graph.py --force-reviewer-issue --turn-ceiling 6
```

Planner node code:

```
def planner_node(state: AgentState) -> Dict[str, Any]:
    """Produce a tag-and-summary proposal, validate it, and record the result."""
    print("--- NODE: Planner ---")

    user_parts = [state["task"]]

    if state["validation_error"]:
        user_parts.append(
            "Your previous answer was rejected by schema validation:\n"
            f"{state['validation_error']}\n"
            "Return a corrected JSON object that fixes exactly that problem."
        )

    issues = (state.get("reviewer_feedback") or {}).get("issues") or []
    if issues:
        user_parts.append(
            "A reviewer raised these issues with your previous answer:\n"
            + "\n".join(f"- {i}" for i in issues)
            + "\nAddress them while keeping exactly 3 tags and a summary of at most 25 words"
        )

    messages = [
        {"role": "system", "content": PLANNER_SYSTEM},
        {"role": "user", "content": "\n\n".join(user_parts)},
    ]

    raw = state["llm"].complete(messages)
    parsed = _parse_json_reply(raw)
```

```

if FORCE_SCHEMA_FAILURE and state["planner_attempts"] == 0 and isinstance(parsed, dict):
    parsed = dict(parsed, tags=(parsed.get("tags") or [])[:2])
    print("    [forced] first attempt truncated to 2 tags")

ok, normalized, error = validate_planner_output(parsed)

if ok:
    proposal = normalized
else:
    # Keep the rejected output so the report can show what was wrong.
    proposal = parsed if isinstance(parsed, dict) else {"raw": str(raw)[:500]}

print(f"    valid={ok}" + (f" error={error}" if error else ""))

return {
    "planner_proposal": proposal,
    "valid": ok,
    "validation_error": error,
    "validation_history": state["validation_history"] + ([error] if error else []),
    "planner_attempts": state["planner_attempts"] + 1,
    "schema_retries": state["schema_retries"] + (0 if ok else 1),
    # Clear stale critique so the Reviewer re-runs against this new proposal.
    "reviewer_feedback": {},
}

```

Supervisor node code:

```

def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """Update state only -- increment the turn counter. No LLM call here."""
    turn = state["turn_count"] + 1
    print(f"--- NODE: Supervisor (turn {turn})/{state['turn_ceiling']} ---")
    return {"turn_count": turn}

def router_logic(state: AgentState) -> str:
    if state["turn_count"] >= state["turn_ceiling"]:
        print(f"    router -> END (hit turn ceiling {state['turn_ceiling']})")
        return END

    if not state["valid"]:
        reason = "no proposal yet" if state["planner_attempts"] == 0 else "schema invalid"
        print(f"    router -> planner ({reason})")
        return "planner"

    feedback = state.get("reviewer_feedback") or {}
    if not feedback:
        print("    router -> reviewer (proposal not yet reviewed)")
        return "reviewer"

    if feedback.get("issues"):
        print(f"    router -> planner ({len(feedback['issues'])} reviewer issues)")
        return "planner"

    print("    router -> END (valid and reviewed clean)")
    return END

```

Planner node code:

```

def planner_node(state: AgentState) -> Dict[str, Any]:
    """Produce a tag-and-summary proposal, validate it, and record the result."""
    print("--- NODE: Planner ---")

    user_parts = [state["task"]]

    if state["validation_error"]:
        user_parts.append(
            "Your previous answer was rejected by schema validation:\n"
            f"{state['validation_error']}\n"
            "Return a corrected JSON object that fixes exactly that problem."
        )

    issues = (state.get("reviewer_feedback") or {}).get("issues") or []
    if issues:
        user_parts.append(
            "A reviewer raised these issues with your previous answer:\n"
            + "\n".join(f"-- {i}" for i in issues)
            + "\nAddress them while keeping exactly 3 tags and a summary of at most 25 words"
        )

    messages = [
        {"role": "system", "content": PLANNER_SYSTEM},
        {"role": "user", "content": "\n\n".join(user_parts)},
    ]

    raw = state["llm"].complete(messages)
    parsed = _parse_json_reply(raw)

    if FORCE_SCHEMA_FAILURE and state["planner_attempts"] == 0 and isinstance(parsed, dict):
        parsed = dict(parsed, tags=(parsed.get("tags") or [])[:2])
        print("    [forced] first attempt truncated to 2 tags")

    ok, normalized, error = validate_planner_output(parsed)

    if ok:
        proposal = normalized
    else:
        proposal = parsed if isinstance(parsed, dict) else {"raw": str(raw)[:500]}

    print(f"    valid={ok}" + (f" error={error}" if error else ""))

    return {
        "planner_proposal": proposal,
        "valid": ok,
        "validation_error": error,
        "validation_history": state["validation_history"] + ([error] if error else []),
        "planner_attempts": state["planner_attempts"] + 1,
        "schema_retries": state["schema_retries"] + (0 if ok else 1),
        "reviewer_feedback": {},
    }

```

Captured output (full transcript in reports/hw02/raw/loop_test_part3_step6.txt):

```
--- NODE: Supervisor (turn 1/6) ---
| router -> planner (no proposal yet)
--- NODE: Planner ---
| [turn 1] input_tokens=153 output_tokens=47 total_tokens=200
|   valid=True
--- NODE: Supervisor (turn 2/6) ---
| router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
| issues=1 (forced)
--- NODE: Supervisor (turn 3/6) ---
| router -> planner (1 reviewer issues)
--- NODE: Planner ---
| [turn 2] input_tokens=202 output_tokens=39 total_tokens=241
|   valid=True
--- NODE: Supervisor (turn 4/6) ---
| router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
| issues=1 (forced)
--- NODE: Supervisor (turn 5/6) ---
| router -> planner (1 reviewer issues)
--- NODE: Planner ---
| [turn 3] input_tokens=202 output_tokens=39 total_tokens=241
|   valid=True
--- NODE: Supervisor (turn 6/6) ---
| router -> END (hit turn ceiling 6)
{
  "valid": true,
  "reviewed_clean": false,
  "succeeded": false,
  "planner_attempts": 3,
  "schema_retries": 0,
  "turn_count": 6,
  "turn_ceiling": 6,
  "hit_ceiling": true,
  "tags": [
    "public transportation delay",
    "bus service disruption",
    "downtown transit issues"
  ],
  "summary": "Bus Line 22 delayed 40 minutes with no prior notice at Downtown Transit Center.",
  "validation_history": [],
  "issue_history": [
    "[forced] Reviewer set to always object (Part 3 Step 6 loop test).",
    "[forced] Reviewer set to always object (Part 3 Step 6 loop test).",
  ],
  "reviewer_issues": [],
  "latency_ms": 30222
}

[exit] turns=3 cumulative_input_tokens=557 cumulative_output_tokens=125 cumulative_total_tokens=682
```

This demonstrates both required behaviours at once: the conditional edge sends work back to the Planner when the Reviewer objects, and the turn ceiling terminates a loop that would otherwise never end.

SCREENSHOT 3.2 -

the correction loop: the Reviewer objects, the router returns control to the Planner, and the turn ceiling stops the loop at turn 6.

```
{
  "dropped": 0
},
"issue_history": []
}
--- NODE: Supervisor (turn 3/4) ---
router -> END (valid and reviewed clean)

{supervisor: {
  "turn_count": 3
}}

[exit] turns=2 cumulative_input_tokens=502 cumulative_output_tokens=51 cumulative_total_tokens=553
(base)
data268-3370 on P main [17] via v3.12.13 (.venv) on (us-east-1) took 12s
> python code/agents_graph.py --force-reviewer-issue --turn-ceiling 6

--- NODE: Supervisor (turn 1/6) ---
router -> planner (no proposal yet)
--- NODE: Planner ---
[turn 1] input_tokens=153 output_tokens=43 total_tokens=196
valid=True
--- NODE: Supervisor (turn 2/6) ---
router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
issues=1 (forced)
--- NODE: Supervisor (turn 3/6) ---
router -> planner (1 reviewer issues)
--- NODE: Planner ---
[turn 2] input_tokens=202 output_tokens=39 total_tokens=241
valid=True
--- NODE: Supervisor (turn 4/6) ---
router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
issues=1 (forced)
--- NODE: Supervisor (turn 5/6) ---
router -> planner (1 reviewer issues)
--- NODE: Planner ---
[turn 3] input_tokens=202 output_tokens=39 total_tokens=241
valid=True
--- NODE: Supervisor (turn 6/6) ---
router -> END (hit turn ceiling 6)
{
  "valid": true,
  "reviewed_clean": false,
  "succeeded": false,
  "planner_attempts": 3,
  "schema_retries": 0,
  "turn_count": 6,
  "turn_ceiling": 6,
  "hit_ceiling": true,
  "tags": [
    "public transportation delay",
    "bus service disruption",
    "downtown transit issues"
  ],
  "summary": "Bus Line 22 delayed 40 minutes with no prior notice at Downtown Transit Center.",
  "validation_history": [],
  "issue_history": [
    "[forced] Reviewer set to always object (Part 3 Step 6 loop test).",
    "[forced] Reviewer set to always object (Part 3 Step 6 loop test).",
  ],
  "reviewer_issues": [],
  "latency_ms": 38441
}

[exit] turns=3 cumulative_input_tokens=557 cumulative_output_tokens=121 cumulative_total_tokens=678
(base)
data268-3370 on P main [17] via v3.12.13 (.venv) on (us-east-1) took 32s
> |
```

Part 4 - Output schema and loop safety

Full tables and analysis in reports/hw02/METRICS.md. Raw data in reports/hw02/raw/.

4.1 Pydantic validation of Planner output

src/schemas.py - exactly three tags of 3–30 characters, summary of at most 25 words:

SCREENSHOT 4.1:

```

class PlannerOutput(BaseModel):
    """Exactly three tags of 3-30 characters each, plus a summary of <=25 words."""

    tags: List[str] = Field(..., min_length=REQUIRED_TAG_COUNT, max_length=REQUIRED_TAG_COUNT)
    summary: str

    @field_validator("tags")
    @classmethod
    def check_tag_lengths(cls, tags: List[str]) -> List[str]:
        for tag in tags:
            n = len(tag.strip())
            if n < TAG_MIN_CHARS or n > TAG_MAX_CHARS:
                raise ValueError(
                    f"tag {tag.strip()!r} is {n} characters; "
                    f"each tag must be {TAG_MIN_CHARS}-{TAG_MAX_CHARS} characters"
                )
        return tags

    @field_validator("summary")
    @classmethod
    def check_summary_length(cls, summary: str) -> str:
        n = len(summary.split())
        if n == 0:
            raise ValueError("summary is empty")
        if n > MAX_SUMMARY_WORDS:
            raise ValueError(f"summary is {n} words; must be at most {MAX_SUMMARY_WORDS}")
        return summary

```

Output is never coerced. This is a deliberate break from HW1's `coerce_reply()`, which silently repairs bad output by padding tags and truncating the summary. Validating coerced output would make every run "valid first attempt" by construction, leaving the retry loop dead and Part 4.3 with nothing to measure.

4.2 Feeding the validation error back

```

if state["validation_error"]:
    user_parts.append(
        "Your previous answer was rejected by schema validation:\n"
        f"{state['validation_error']}\n"
        "Return a corrected JSON object that fixes exactly that problem."
    )

```

The error text names the field and the fault, so the retry corrects a specific defect rather than re-rolling the dice.

On this input the model never actually violates the schema (see 4.3), so the retry path is demonstrated with a flag that corrupts the first Planner attempt - the same technique the spec prescribes for the Step 6 loop test:

```
python code/agents_graph.py --force-schema-failure --turn-ceiling 8
```

Captured output (full transcript in reports/hw02/raw/retry_test_part4_2.txt):

```
--- NODE: Supervisor (turn 1/8) ---
| router -> planner (no proposal yet)
--- NODE: Planner ---
[turn 1] input_tokens=153 output_tokens=47 total_tokens=200
| [forced] first attempt truncated to 2 tags
| valid=False error=tags: List should have at least 3 items after validation, not 2
--- NODE: Supervisor (turn 2/8) ---
| router -> planner (schema invalid)
--- NODE: Planner ---
[turn 2] input_tokens=190 output_tokens=45 total_tokens=235
| valid=True
--- NODE: Supervisor (turn 3/8) ---
| router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
[turn 3] input_tokens=348 output_tokens=51 total_tokens=399
| issues=1
--- NODE: Supervisor (turn 4/8) ---
| router -> planner (1 reviewer issues)
--- NODE: Planner ---
[turn 4] input_tokens=209 output_tokens=41 total_tokens=250
| valid=True
--- NODE: Supervisor (turn 5/8) ---
| router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
[turn 5] input_tokens=347 output_tokens=8 total_tokens=355
| issues=0
--- NODE: Supervisor (turn 6/8) ---
| router -> END (valid and reviewed clean)
```

```

{
  "valid": true,
  "reviewed_clean": true,
  "succeeded": true,
  "planner_attempts": 3,
  "schema_retries": 1,
  "turn_count": 6,
  "turn_ceiling": 8,
  "hit_ceiling": false,
  "tags": [
    "Bus Delay",
    "Downtown Wait",
    "No Announcement"
  ],
  "summary": "Bus Line 22 delayed 40 minutes with no prior notice at Downtown Transit Center.",
  "validation_history": [
    "tags: List should have at least 3 items after validation, not 2"
  ],
  "issue_history": [
    "[rule 1] 'Transit Center': The tag 'Transit Center' is about a subject the input never raises."
  ],
  "reviewer_issues": [],
  "latency_ms": 78513
}

[exit] turns=5 cumulative_input_tokens=1247 cumulative_output_tokens=192 cumulative_total_tokens=1439

```

Both correction mechanisms are visible in one run: the schema rejection at turn 2 and the Reviewer objection at turn 4, each routed back to the Planner and each fixed, all within the turn ceiling.

SCREENSHOT 4.2 - schema rejection at turn 2 fed back to the Planner and corrected at turn 3, then a Reviewer objection at turn 4 handled the same way.

```

datax60-3178 on y asah 174 via 45 V3.12.13 (.venv) on 45 (us-east-1) took 32s
python code/agents_graph.py --force-schema-failure --turn-ceiling 8

--- NODE: Supervisor (turn 1/8) ---
router -> planner (no proposal yet)
--- NODE: Planner ---
[turn 1] input_tokens=153 output_tokens=47 total_tokens=200
[forced] first attempt truncated to 2 tags
valid=False error=tags: List should have at least 3 items after validation, not 2
--- NODE: Supervisor (turn 2/8) ---
router -> planner (schema invalid)
--- NODE: Planner ---
[turn 2] input_tokens=190 output_tokens=45 total_tokens=235
valid=True
--- NODE: Supervisor (turn 3/8) ---
router -> reviewer (proposal not yet reviewed)
--- NODE: Reviewer ---
[turn 3] input_tokens=348 output_tokens=8 total_tokens=356
issues=0
--- NODE: Supervisor (turn 4/8) ---
router -> END (valid and reviewed clean)
{
  "valid": true,
  "reviewed_clean": true,
  "succeeded": true,
  "planner_attempts": 2,
  "schema_retries": 1,
  "turn_count": 4,
  "turn_ceiling": 8,
  "hit_ceiling": false,
  "tags": [
    "Bus Delay",
    "Transit Center",
    "Service Disruption"
  ],
  "summary": "Bus Line 22 delayed 40 minutes at Downtown Transit Center with no prior notice.",
  "validation_history": [
    "tags: List should have at least 3 items after validation, not 2"
  ],
  "issue_history": [],
  "reviewer_issues": [],
  "latency_ms": 27127
}
[exit] turns=3 cumulative_input_tokens=691 cumulative_output_tokens=100 cumulative_total_tokens=791

```

4.3 Thirty runs on the frozen input

Input: reports/hw02/cases/schema_input.json. Turn ceiling 8.

4.3 Schema validation — 30 runs, ceiling 8



#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
1	valid first attempt	3	0	7	no	65,301
2	valid first attempt	3	0	7	no	63,689
3	valid first attempt	3	0	7	no	61,761
4	valid first attempt	3	0	7	no	61,040
5	valid first attempt	3	0	7	no	60,575
6	valid first attempt	3	0	7	no	61,880
7	valid first attempt	3	0	7	no	61,432
8	valid first attempt	3	0	7	no	60,876
9	valid first attempt	3	0	7	no	62,100
10	valid first attempt	3	0	7	no	61,591
11	valid first attempt	3	0	7	no	60,643
12	valid first attempt	3	0	7	no	62,870
13	valid first attempt	3	0	7	no	63,530

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
14	valid first attempt	3	0	7	no	62,119
15	valid first attempt	3	0	7	no	62,070
16	valid first attempt	3	0	7	no	62,061
17	valid first attempt	3	0	7	no	62,391
18	valid first attempt	3	0	7	no	62,525
19	valid first attempt	3	0	7	no	62,692
20	valid first attempt	3	0	7	no	62,038
21	valid first attempt	3	0	7	no	61,480
22	valid first attempt	3	0	7	no	62,778
23	valid first attempt	3	0	7	no	62,933
24	valid first attempt	3	0	7	no	62,315
25	valid first attempt	3	0	7	no	62,295
26	valid first attempt	3	0	7	no	62,394
27	valid first attempt	3	0	7	no	62,882
28	valid first attempt	3	0	7	no	62,668
29	valid first attempt	3	0	7	no	62,665
30	valid first attempt	3	0	7	no	62,215

30 runs · mean latency 62,260 ms · 0 reached the ceiling

Outcome over 30 runs	Count	Mean latency (ms)
Valid first attempt	30	62,260
Valid after 1 retry	0	-
Valid after 2+ retries	0	-
Hit turn ceiling	0	-

Latency spread: min 60,575 ms, median 62,255 ms, max 65,301 ms.

Pydantic rejected nothing in 30 runs, so the schema retry path is not exercised here. At temperature 0 on an ordinary single-topic incident, qwen3:8b reliably produces three tags

and a short summary - HW1's non-determinism experiment found the same determinism (1 distinct tag set across 20 runs at temperature 0.0).

The runs were not short, however: each took 7 turns and 3 Planner attempts, because the *Reviewer* returned the proposal twice per run. Those are content objections, not schema failures, and they are counted separately. Section 4.6 examines them - they turn out to be wrong, and that is the most significant result in this assignment.

SCREENSHOT 4.3 - the full experiment summary rebuilt from all 75 saved runs.

```
data268-3170 on / main [??] via 3 v3.12.13 (.venv) on (us-east-1) took 28s
> python code/run_schema_experiments.py --summarize-only

{
  "model": "qwen3:8b",
  "rebuilt_from": "reports/hw02/raw/runs.jsonl",
  "rebuilt_at": "2026-09-13T01:35:12Z",
  "total_runs": 75,
  "schema_validation": {
    "runs": 30,
    "turn_ceiling": 0,
    "outcomes": {
      "valid_first_attempt": {
        "count": 30,
        "mean_latency_ms": 62268.3
      },
      "valid_after_1_retry": {
        "count": 0,
        "mean_latency_ms": null
      },
      "valid_after_2plus_retries": {
        "count": 0,
        "mean_latency_ms": null
      },
      "hit_turn_ceiling": {
        "count": 0,
        "mean_latency_ms": null
      }
    }
  },
  "ceiling_comparison": {
    "2": {
      "runs": 20,
      "completed": 0,
      "completion_rate": 0.0,
      "mean_latency_ms": 9822.3,
      "median_latency_ms": 8977.0,
      "mean_planner_attempts": 1
    },
    "10": {
      "runs": 20,
      "completed": 20,
      "completion_rate": 1.0,
      "mean_latency_ms": 61167.6,
      "median_latency_ms": 68959.5,
      "mean_planner_attempts": 3
    }
  },
  "adversarial": {
    "case_id": "adversarial_input",
    "runs": 5,
    "hit_ceiling_count": 5,
    "hit_ceiling_rate": 1.0,
    "outcomes": {
      "valid_first_attempt": {
        "count": 5,
        "mean_latency_ms": 149383.4
      },
      "valid_after_1_retry": {
        "count": 0,
        "mean_latency_ms": null
      },
      "valid_after_2plus_retries": {
        "count": 0,
        "mean_latency_ms": null
      },
      "hit_turn_ceiling": {
        "count": 0,
        "mean_latency_ms": null
      }
    }
  }
}

(adbase)
data268-3170 on / main [??] via 3 v3.12.13 (.venv) on (us-east-1)
> |
```

4.4 Turn ceiling 2 versus 10

Same frozen input and model settings, 20 runs each.

Turn ceiling	Runs	Completed	Completion rate	Mean latency (ms)	Turns used	Planner attempts
2	20	0	0.0%	9,022	2	1
10	20	20	100.0%	61,168	7	3

Ceiling 2 never completes, structurally. The supervisor increments the counter before every routing decision, so a ceiling of 2 buys exactly one worker turn: the Planner runs, the counter reaches 2, and the router stops the graph before the Reviewer has ever seen the proposal. The output is schema-valid but unreviewed. The rate is 0/20 rather than merely low because completion is impossible by construction.

Please refer next page for result:

4.4 Ceiling 2 — 20 runs

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
1	valid first attempt	1	0	2	yes	9,831
2	valid first attempt	1	0	2	yes	8,826
3	valid first attempt	1	0	2	yes	8,971
4	valid first attempt	1	0	2	yes	9,121
5	valid first attempt	1	0	2	yes	8,829
6	valid first attempt	1	0	2	yes	8,954
7	valid first attempt	1	0	2	yes	8,882
8	valid first attempt	1	0	2	yes	8,990
9	valid first attempt	1	0	2	yes	8,891
10	valid first attempt	1	0	2	yes	8,991
11	valid first attempt	1	0	2	yes	8,976
12	valid first attempt	1	0	2	yes	9,196
13	valid first attempt	1	0	2	yes	9,070
14	valid first attempt	1	0	2	yes	8,978
15	valid first attempt	1	0	2	yes	9,032
16	valid first attempt	1	0	2	yes	8,942
17	valid first attempt	1	0	2	yes	9,051
18	valid first attempt	1	0	2	yes	8,929

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
19	valid first attempt	1	0	2	yes	8,939
20	valid first attempt	1	0	2	yes	9,134

20 runs · mean latency 9,022 ms · 20 reached the ceiling

Choice for deployment: ceiling 10. Every completed run used exactly 7 turns, so the minimum workable ceiling for this input is 7 - ceilings of 2, 4 and 6 would all fail. Ten is the smallest offered option that clears 7 with margin for one more correction round.

4.4 Ceiling 10 — 20 runs

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
1	valid first attempt	3	0	7	no	62,500
2	valid first attempt	3	0	7	no	63,193
3	valid first attempt	3	0	7	no	62,522
4	valid first attempt	3	0	7	no	60,449
5	valid first attempt	3	0	7	no	61,285
6	valid first attempt	3	0	7	no	61,887
7	valid first attempt	3	0	7	no	60,5
8	valid first attempt	3	0	7	no	61,796
9	valid first attempt	3	0	7	no	61,363
10	valid first attempt	3	0	7	no	60,466
11	valid first attempt	3	0	7	no	61,043
12	valid first attempt	3	0	7	no	60,537
13	valid first attempt	3	0	7	no	60,876
14	valid first attempt	3	0	7	no	60,499

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
15	valid first attempt	3	0	7	no	60,492
16	valid first attempt	3	0	7	no	61,057
17	valid first attempt	3	0	7	no	61,363
18	valid first attempt	3	0	7	no	60,794
19	valid first attempt	3	0	7	no	60,613
20	valid first attempt	3	0	7	no	60,101

20 runs · mean latency 61,167 ms · 0 reached the ceiling

The cost should be stated plainly: 61,168 ms against 9,022 ms, about **6.8×** the latency. That is not the price of the ceiling itself but of letting the Reviewer loop run to convergence - ceiling 2 is cheap precisely because it performs no review, which is the same reason it never completes. Between a fast unreviewed answer and a slow reviewed one, the reviewed one is deployable; but section 4.6 shows most of that 6.8× is spent on objections that should never have been raised, so the honest recommendation is ceiling 10 **together with** the fix proposed in 4.5.

SCREENSHOT 4.4 - ceiling comparison: 0% completion at ceiling 2 against 100% at ceiling 10.

```
(base)
data260-3170 on y main [??] via v3.12.13 (.venv) on us-east-1
python code/run_schema_experiments.py --summarize-only | python -c "import json,sys; d=json.load(sys.stdin); print(json.dumps({'ceiling_comparison':d['ceiling_comparison']}, indent=2))"
{
  "ceiling_comparison": {
    "2": {
      "runs": 20,
      "completed": 0,
      "completion_rate": 0.0,
      "mean_latency_ms": 9022.3,
      "median_latency_ms": 8977.0,
      "mean_planner_attempts": 1
    },
    "10": {
      "runs": 20,
      "completed": 20,
      "completion_rate": 1.0,
      "mean_latency_ms": 61167.6,
      "median_latency_ms": 60959.5,
      "mean_planner_attempts": 3
    }
  }
}
```

4.5 Adversarial input

reports/hw02/cases/adversarial_input.json. Turn ceiling 10, 5 runs.

Metric	Result
Runs reaching the turn ceiling	5 / 5 (100%)
Turns used per run	10, 10, 10, 10, 10
Planner attempts per run	5, 5, 5, 5, 5
Schema retries per run	0, 0, 0, 0, 0
Mean latency	149,383 ms
Versus the frozen input	2.4× slower

4.5 Adversarial — 5 runs, ceiling 10

#	Outcome	Attempts	Retries	Turns	Hit ceiling	Latency (ms)
1	valid first attempt	5	0	10	yes	162,970
2	valid first attempt	5	0	10	yes	147,042
3	valid first attempt	5	0	10	yes	145,975
4	valid first attempt	5	0	10	yes	145,635
5	valid first attempt	5	0	10	yes	145,295

5 runs · mean latency 149,383 ms · 5 reached the ceiling

Why does it cause trouble? The input is a quarter-end digest spanning about ten distinct sub-incidents. The schema allows 25 words. Ten sub-incidents cannot be stated in 25 words, so the Planner must generalise - *"Multiple transit routes and subsystems faced delays, equipment outages, and vendor ticket issues with escalation."* The Reviewer then objects under rule 2, that the summary asserts something the input does not state, because no sentence in the input says "multiple subsystems faced outages"; the input lists them individually.

The two constraints are in direct conflict: the word limit compels abstraction and the faithfulness rule forbids it. All 20 objections across the 5 runs were rule 2, and the final tag set was identical in every run - a reproducible structural failure, not noise.

Proposed fix - a no-progress circuit breaker. Hash each Reviewer objection set; if the same objection is raised twice in a row, the loop is not converging and further turns cannot help. Stop and return the current schema-valid proposal, flagged "accepted without Reviewer sign-off". On this data that would cut the adversarial case from ~149s to roughly 60s, and the ordinary frozen input from ~62s to ~40s, since section 4.6 shows its two objections are the same complaint repeated.

SCREENSHOT 4.5 - the adversarial section of the console summary, showing 5/5 reaching the ceiling.

```
data260-3170 on / main [??] via v3.12.13 (.venv) on (us-east-1)
> python code/run_schema_experiments.py --summarize-only | python -c "import json,sys; d=json.load(sys.stdin); print(json.dumps({'adversarial':d['adversarial']}), indent=2)"

{
  "adversarial": {
    "case_id": "adversarial_input",
    "runs": 5,
    "hit_ceiling_count": 5,
    "hit_ceiling_rate": 1.0,
    "outcomes": {
      "valid_first_attempt": {
        "count": 5,
        "mean_latency_ms": 149383.4
      },
      "valid_after_1_retry": {
        "count": 0,
        "mean_latency_ms": null
      },
      "valid_after_2plus_retries": {
        "count": 0,
        "mean_latency_ms": null
      },
      "hit_turn_ceiling": {
        "count": 0,
        "mean_latency_ms": null
      }
    }
  }
}
(base)
```

4.6 An unplanned finding: the Reviewer is prompt-format sensitive

Two full 75-run batches were collected. Between them, **one thing changed in the Reviewer's input: how the proposal was formatted**. Batch 1 passed it as `json.dumps(proposal, indent=2)`; batch 2 passed it as plain text (Tags: a; b; c\nSummary: ...). Same model, same temperature, same input, same rules, same validator, same machine.

	Batch 1 (JSON framing)	Batch 2 (plain-text framing)
Reviewer objections across 30 runs	0	60
Turns per run	3	7
Planner attempts per run	1	3
Mean latency, frozen input	10,862 ms	62,260 ms
Completion rate, ceiling 10	20/20	20/20
Completion rate, ceiling 2	0/20	0/20
Adversarial ceiling hits	5/5	5/5

Every structural result reproduced exactly - completion rates, ceiling behaviour and adversarial outcomes are identical, which is direct evidence that those findings are robust rather than artefacts of one run. What changed by a factor of six is latency, and it changed entirely because of the Reviewer's objection rate.

The objections were wrong. All 60 were the same two:

rule 1 'Transit Center': The tag 'Transit Center' is about a subject the input never raises.

rule 1 'Bus Delay': The tag 'Bus Delay' is about a subject the input never raises.

The input reads *"Delayed Line 22 bus at Downtown Transit Center ... the 7:15am bus never arrived."* Both tags are plainly supported. The Reviewer's own prompt names these exact cases as acceptable - *"'Transit Center' for 'Downtown Transit Center' ... must not be reported"* - so the model was told, in the same prompt, not to make the objection it then made 60 times. The loop still converged, because the Planner swapped the rejected tags for wordier synonyms until the Reviewer stopped objecting. The six-fold latency bought nothing.

What this means. An instruction-following failure this specific cannot be fixed by adding more instruction; the counter-example was already present and was ignored. Two conclusions follow:

1. **Prompt formatting is a live variable, not presentation.** A change most engineers would treat as cosmetic flipped the objection rate from 0% to 100%. Measurements taken across a prompt change are not comparable - which is why both batches are retained rather than only the newer one.
2. **Enforce contracts in code, not in prose.** These objections satisfied the ReviewIssue contract: well-formed, correctly quoting real text. They were merely wrong. That is the limit of what a schema can catch, and it is why the circuit breaker proposed in 4.5 is the right defence - it does not require the Reviewer to be correct, only requires noticing the loop has stopped making progress.

JSON framing was not simply the better of the two: under it the Reviewer objected to the { and } of the serialisation itself under rule 3 (see raw/discarded_adversarial_prompt_bug.jsonl). Both framings produce spurious objections; they differ in which ones, and on which input.

Verification

python code/verify_hw02.py → reports/hw02/verification.json

SCREENSHOT 5.1:

```
}
}
(base)
data260-3170 on 7 main [??] via 3 v3.12.13 (.venv) on 4 (us-east-1)
> python code/verify_hw02.py

=== Static files ===
(PASS) file_exists:html -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/code/web_application/HW1-PragnayaPriyadarshini.html
(PASS) file_exists:styles_css -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/code/web_application/styles.css
(PASS) file_exists:hw1.js -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/code/web_application/hw1.js
(PASS) css:responsive_breakpoint -- covers 375px
(PASS) css:wraps_long_text -- prevents horizontal scroll at 375px
(PASS) css:state-loading_styled
(PASS) css:state-empty_styled
(PASS) css:state-error_styled
(PASS) file_exists:domain_schema -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/DOMAIN_SCHEMA.md
(PASS) file_exists:requirements -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/requirements.txt
(PASS) compile:model_client
(PASS) compile:schemas
(PASS) compile:run_schema_experiments

=== Part 3 model-client routing ===
(PASS) routing:no_direct_model_calls
(PASS) routing:all_llm_calls_via_model_client -- 2/2 call sites use state['llm'].complete()

=== Part 4 schema validator ===
(PASS) schemas:importable
(PASS) schemas:accepts_valid
(PASS) schemas:rejects_two_tags
(PASS) schemas:rejects_four_tags
(PASS) schemas:rejects_short_tag
(PASS) schemas:rejects_long_summary

=== Ollama ===
(PASS) ollama:reachable
(PASS) ollama:model_pulled -- ['smollm1.7b', 'qwen3:8b']

=== Part 2 FastAPI ===
(PASS) file_exists:app.py -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/code/web_application/app.py
(PASS) compile:app.py
(PASS) fastapi:port_free -- PORT_BASE=8470
(PASS) fastapi:starts_on_port_base
(PASS) fastapi:get_list_200 -- status=200
(PASS) fastapi:get_list_returns_array -- typelist
(PASS) fastapi:post_create_201 -- status=201
(PASS) fastapi:post_returns_id -- 409
(PASS) fastapi:list_grew_by_one -- 2 -> 3
(PASS) fastapi:put_update_200 -- status=200
(PASS) fastapi:update_persisted -- routeId=3170-updated-route
(PASS) fastapi:search_filters -- status=200 hits=2
(PASS) fastapi:delete_2xx -- status=200
(PASS) fastapi:list_shrank_by_one -- 3 -> 2

=== Part 3/4 graph ===
(PASS) file_exists:agents_graph -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/code/agents_graph.py
(PASS) compile:agents_graph
(PASS) file_exists:schema_input -- /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/reports/hw02/cases/schema_input.json
(PASS) graph:completes_without_hanging -- 50285 ms
(PASS) graph:returns_result
(PASS) graph:respected_turn_ceiling -- turn_count=4 ceiling=4
(PASS) graph:made_at_least_one_attempt -- attempts=2
(PASS) graph:valid_run_has_exactly_3_tags -- tags=['Bus Delay', 'Waiting Time', 'No Announcement']
(PASS) graph:valid_run_summary_within_25_words -- words=17

46/46 checks passed
Written to: /Users/ritchit/Desktop/DATA 260--Agentic AI/data260-3170/reports/hw02/verification.json
(base)
data260-3170 on 7 main [??] via 3 v3.12.13 (.venv) on 4 (us-east-1) took 52s
>
```

The smoke test starts the FastAPI backend on PORT_BASE and exercises it over HTTP, then runs the graph once under a timeout. Every check is behavioural, never a text match - it asserts "did it return exactly 3 tags?" and "did the request succeed?", not specific wording, because the model never says the same thing twice.

46 / 46 checks passed.

Reproducing

```
source .venv/bin/activate
pip install -r requirements.txt
```

```
# Parts 1 & 2 - front end + API on PORT_BASE 8470
cd code/web_application && uvicorn app:app --port 8470
```

```
# Part 3 - one graph run, streaming each node's state update
python code/agents_graph.py --stream
python code/agents_graph.py --force-reviewer-issue --turn-ceiling 6 # Step
```

```
6 Loop test
python code/agents_graph.py --force-schema-failure --turn-ceiling 8 # 4.2
retry demo
```

```
# Part 4 - all 75 runs (measured: 66 minutes on this hardware)
caffeinate -i python code/run_schema_experiments.py 2>&1 | tee
reports/hw02/RUN_LOG.txt
python code/run_schema_experiments.py --summarize-only # rebuild tables
from saved runs
# Verification
python code/verify_hw02.py
```

Table of content of where the experiment data lives:

What	Where
Every run individually (all 75)	Sections 4.3–4.5 of this report
Real console output with timestamps	reports/hw02/RUN_LOG.txt — 2,136 lines
Machine-readable per-run records	reports/hw02/raw/runs.jsonl, raw/schema_runs.csv
Computed summary tables	reports/hw02/raw/schema_summary.json
Frozen inputs used	reports/hw02/cases/

Use of AI assistance

Please refer to reports/hw02/AI_USE.md for AI usage declarations.